Experiential Learning
Project Assessment-I on

"Expression Tree Generator and Evaluator"

under the course
Python Programming Language (N-PCCCM304P)

Presented By
Pranjal Parihar
USN No : CM25101
Sec : B

Under the Guidance
of
Mrs. Mayuri Getme

Session:2026-27 (ODD)
Department of CSE(AI&ML)
S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH,NAGPUR
An Autonomous Institute, Affiliated to RTMNU, Nagpur

PROJECT OVERVIEW

Problem Statement

Manual evaluation of arithmetic expressions is error-prone

Objectives

Define project goals and success metrics

Algorithm

Core logical approach for tree construction & evaluation

Flowchart / Pseudocode

Logical flow and recursive traversal logic

Python Logic

Core classes, functions and recursive tree operations

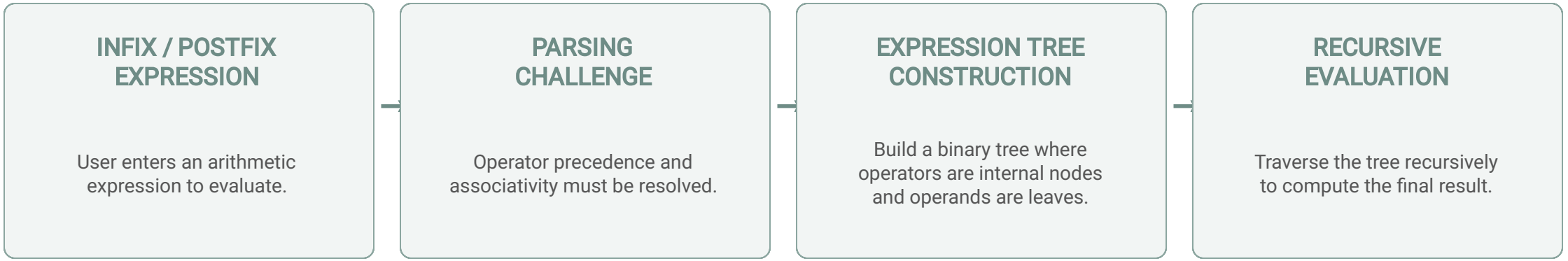
GitHub

Project repository, commit history, and results

PROBLEM STATEMENT & OBJECTIVES

Expression Tree Generator and Evaluator

- **Evaluating complex arithmetic expressions manually is error-prone and does not scale to nested operations.**
- **The project constructs an expression tree from an arithmetic expression and evaluates it using recursive tree traversal.**



"Construct, traverse, and evaluate arithmetic expressions using tree data structures."

ALGORITHM & FLOWCHART

- **The algorithm defines the step-by-step process from expression input to the final evaluated result.**
- **It includes tokenizing, tree construction, recursive traversal, and evaluation.**

ALGORITHM

01 Input arithmetic expression

02 Tokenize expression

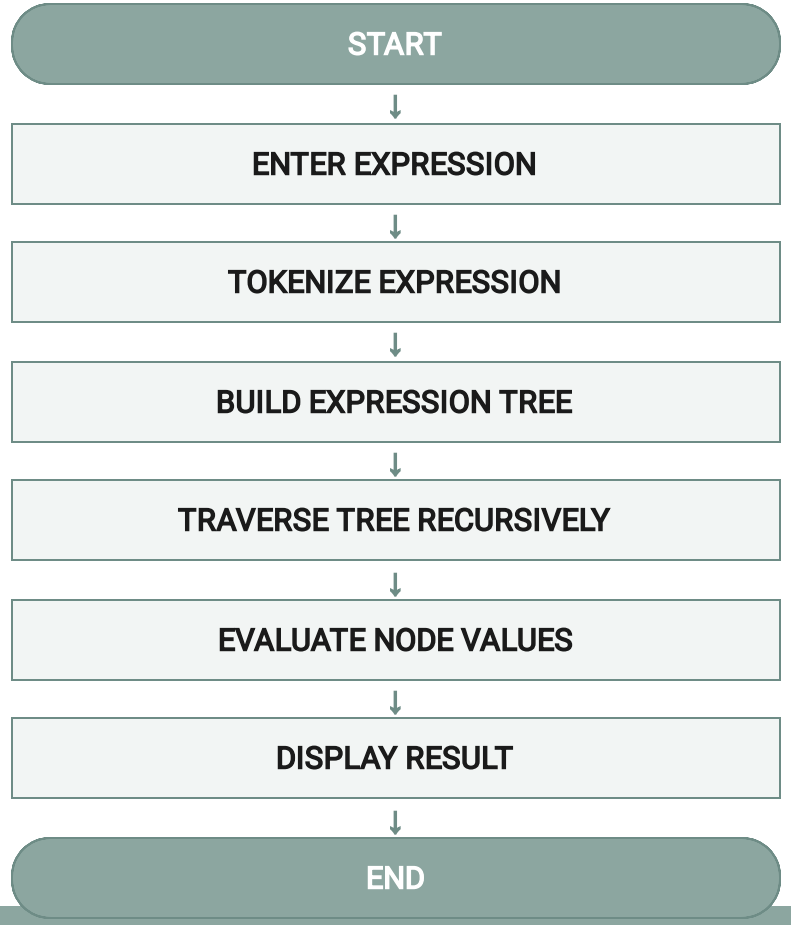
03 Build expression tree (stack-based)

04 Traverse tree recursively (in/post-order)

05 Evaluate node values recursively

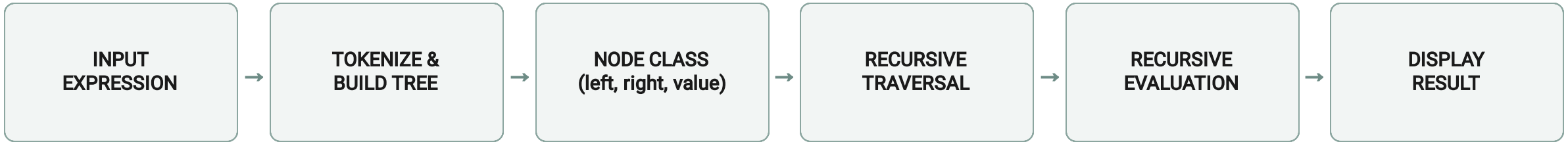
06 Display final result

FLOWCHART



UNDERSTANDING OF PYTHON LOGIC

- Python is used for parsing, tree-node representation, recursive traversal, and evaluation.
- The program uses classes, recursion, and stack-based construction to build and evaluate the tree.



Technologies/Concepts Used

- Python
- Object-Oriented Programming (classes)
- Recursion
- Stack (for tree construction)
- Tree traversal (inorder / postorder)

PHASE-II: CODING & IMPLEMENTATION

Technology Used

Python

Implementation

- Expression is taken as user input.
- Expression is tokenized into operands and operators.
- A Node class stores value, left child, and right child.
- Expression tree is built using a stack-based approach.
- Tree is traversed recursively (inorder / postorder).
- Each node is evaluated recursively to compute the result.
- Final evaluated result is displayed.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def build_tree(postfix):
    stack = []
    for token in postfix:
        node = Node(token)
        if token in '+-*/':
            node.right = stack.pop()
            node.left = stack.pop()
        stack.append(node)
    return stack.pop()

def evaluate(node):
    if node.left is None and node.right is None:
        return float(node.value)
    left_val = evaluate(node.left)
    right_val = evaluate(node.right)
    if node.value == '+': return left_val + right_val
    if node.value == '-': return left_val - right_val
    if node.value == '*': return left_val * right_val
    if node.value == '/': return left_val / right_val

expr = input('Enter postfix expression: ').split()
tree = build_tree(expr)
print('Result:', evaluate(tree))
```

PHASE-II: PROGRAM OUTPUT

Working Flow

Enter Expression

Tokenize Expression

Build Expression Tree

Traverse Tree Recursively

Evaluate Nodes Recursively

Display Output

Sample Output

```
Enter postfix expression: 3 4 + 2 *
Result: 14.0

Enter postfix expression: 5 1 2 + 4 * + 3 -
Result: 14.0

Enter postfix expression: 9 3 /
Result: 3.0
```

Implemented Features

- Expression tree construction
- Recursive inorder/postorder traversal
- Recursive expression evaluation
- Console-based input and output

PHASE-III: TESTING & RESULTS

Test Cases (Postfix → Result)

3 4 + 2 * → 14.0

5 1 2 + 4 * + 3 - → 14.0

9 3 / → 3.0

8 2 / 3 - → 1.0

All test cases matched the expected (manual) results.

Test Output

```
Enter postfix expression: 3 4 + 2 *
Result: 14.0

Enter postfix expression: 8 2 / 3 -
Result: 1.0

Enter postfix expression: 9 3 /
Result: 3.0
```

Observations

- **Nested expressions evaluate correctly using recursion**
- **Tree structure removes precedence errors**
- **Division by zero / invalid input not handled yet**

PHASE-III: GITHUB, CONCLUSION & FUTURE SCOPE

GitHub Repository

- **Link: github.com/<your-username>/expression-tree-evaluator**
- **expression_tree.py: Node class, build_tree(), evaluate()**
- **README.md: problem statement, usage and sample output**
- **Commit history showing phase-wise development**

Conclusion

- **Expression trees with recursion make evaluation simple and accurate**
- **The project applies OOP, stack and recursion in Python**

Future Scope

- **Infix input via Shunting-Yard**
- **Parentheses and exponent (^)**
- **Division-by-zero handling**
- **Graphical tree visualisation**
- **GUI version**

Thank You

Pranjal Parihar--CM25101